

Capabilities — Proof-Based Vulnerability Coverage

Shannon performs real, automated exploitation and only records a finding when it is confirmed by a benign proof signal — arithmetic evaluation, an out-of-band callback, a database error, a browser-executed script, a differential leak, or a file read. Zero false positives by construction.

27

PROOF-BASED CLASSES

0

FALSE POSITIVES (BY CONSTRUCTION)

Exploit → Defend → Prove → Report

EVERY CONFIRMED FINDING

✓ Proof engine — the truth

Deterministic probers send real crafted requests and confirm with a benign marker. If Shannon says **CONFIRMED**, it is real — this layer carries zero false positives.

◆ AI layer — the judgment

LLM agents reason about the app and adversarially **judge** findings to cut noise (refute-only — they can never invent a finding). Smart, but judgment: treat AI "potential" items as leads.

Vulnerability classes & what each PROVES

Injection

CLASS	WHAT SHANNON PROVES (THE MARKER)	SEVERITY
SQL injection	Error-based (DB error absent from baseline), boolean-blind (TRUE≠FALSE response), and time-based (injected sleep delays the response and scales with the delay).	Critical
SQLi auth-bypass	Actually logs in via ' OR '1'='1' --: a TRUE tautology gets a session that a FALSE tautology and bad credentials do not.	Critical
NoSQL injection	MongoDB operator injection: key[\$ne] vs key[\$eq] changes the query result set (reproduced).	Critical
Command injection	The shell evaluates an injected echo \$((7*13)) → 91.	Critical
Server-side template injection	{{9931*9817}} evaluates to 97492627 (product can't appear by chance; operand absent).	Critical
Insecure deserialization	A benign pickle payload evaluates a high-entropy arithmetic product in the response.	Critical
XXE	The XML parser resolves an external entity to our out-of-band listener (callback received).	High
CRLF / response splitting	An injected %0d%0a creates an attacker-controlled response header.	High
Host header injection	An attacker Host is reflected into an absolute link/redirect (password-reset & cache poisoning).	Medium
XSS — reflected	The injected JavaScript actually executes in a real browser (unique-nonce callback), not just reflects.	Medium
XSS — stored	A payload submitted to one form executes when a <i>different</i> page is rendered (persistence proven).	High

XSS — DOM-based	A client-side sink (query or URL fragment) executes a payload the server never reflected.	High
-----------------	---	------

Broken access control & authentication

CLASS	WHAT SHANNON PROVES (THE MARKER)	SEVERITY
BOLA / IDOR (2-session)	User A's session reads User B's private data — most of B's private tokens leak to A, absent from the public view. Peer-to-peer only.	High
BFLA (2-session)	A low-privilege session invokes an admin-only function and receives the admin-only content.	Critical
IDOR (single-session)	Distinct per-record secrets returned for different object IDs without authorization (invalid-ID control).	High
Weak / default credentials	A default credential pair actually logs in (while known-bad credentials fail).	Critical
Missing rate-limiting	Many rapid failed logins with no lockout / 429 / CAPTCHA — brute-force is feasible.	Medium
Username enumeration	Login responses differ for existing vs non-existing accounts (gated by a determinism control).	Medium
JWT / token forgery	A token signed with a weak secret is accepted; a forged admin token unlocks content unavailable without it.	High

Server-side, request & client-side

CLASS	WHAT SHANNON PROVES (THE MARKER)	SEVERITY
SSRF	The server fetches an attacker-supplied URL — out-of-band callback received.	High
Path traversal / LFI	Reads a system file it shouldn't (e.g. /etc/passwd signature).	Critical
Open redirect	Redirects to an attacker-controlled host (destination host = attacker domain).	Low
CORS misconfiguration	An arbitrary Origin is reflected <i>with credentials</i> — any site can read authenticated responses.	High
CSRF	A state-changing form with no anti-CSRF token and a SameSite=None session cookie (forgeable cross-site).	Medium
Mass assignment	An unexpected privileged field (role / isAdmin) is bound — proven selectively (a control field is not).	High
Prompt injection (LLM)	The model follows an injected instruction (returns a transformed nonce oracle).	High

Information disclosure & misconfiguration

CLASS	WHAT SHANNON PROVES (THE MARKER)	SEVERITY
Secrets exposure	A sensitive file is served (.env, .git, backups) confirmed by content signature + content-type guard.	High
Verbose errors	Malformed input leaks a stack trace / debug page / server file path absent from the baseline.	Medium
GraphQL introspection	The schema is disclosed to unauthenticated clients (parsed __schema).	Medium
Security headers	Missing/weak CSP, HSTS, X-Content-Type-Options, X-Frame-Options, cookie flags.	Low
Exposure templates	Data-driven Nuclei-style checks (exposed phpinfo, git HEAD, actuator/env, directory listing, ...).	Medium

How every confirmed finding is handled

Exploit (real request + benign marker) → **Defend** (a detection rule + a concrete code fix, and for payload classes a live filtering proxy that re-runs the attack and proves it is now blocked with a 403) → **Report** (CVSS 3.1 score + vector, mapped to OWASP WSTG/ASVS, OWASP Top 10, CWE, MITRE ATT&CK, PCI DSS 4.0 and ISO 27001, with evidence and a sign-off block).

Straight talk (what "real" means here). The **proof engine is real and deterministic** — a CONFIRMED finding is backed by a real request and a benign marker, not a guess. The **AI layer is real software but is judgment, not proof** — treat its "potential" items as leads. Coverage is focused, not exhaustive: business-logic flaws, deep exploit chaining, and HTTP request smuggling are out of scope. Shannon is not a replacement for a full manual penetration test, and a report becomes a **certified** pentest only once a certified human (OSCP / CREST) reviews and signs it. We pair the tool with that sign-off — accuracy you can trust, honestly stated.